Specification: CARRT Robotic Assistive Arm Project

Oliver Gaston
Brielle Ashmeade
Allison Moline
Jadon Orr

Bellini College of AI, Cybersecurity and Computing
University of South Florida
Tampa, FL 33620

Email:
gaston13@usf.edu
ashmeadeb@usf.edu
allison439@usf.edu
jjorr@usf.edu

Version 3.0
March 6, 2026

History of Specification Document

- **Version 1.00** (March 3, 2026) – Initial document created
- **Version 2.00** (March 5, 2026) – Added to document sections: Design, Constraints, and Tradeoffs.
- **Version 3.00** (March 6, 2026) – Completed and refined document for submission.

Table of Contents

1. Introduction 1

2. Glossary 1-2

3. Constraints 3-4

4. Applicable standards 4

5. Design 4-10

6. Risk analysis and mitigation 10-11

7. Design trade-offs 11-12

8. Project plan 12-14

9. Traceability to requirements 14-15

10. Traceability to needs and factors 16-17

References 18

1. Introduction

The purpose of this document is to describe the design specifications for the CARRT Assistive Arm Project. The main goal of this project is to develop an assistive robotic system capable of helping individuals with limited upper-limb mobility perform Activities of Daily Living (ADLs).

The system uses a Kinova Gen3 robotic arm mounted on a wheelchair and controlled through a ROS 2-based software architecture, with an integrated camera that allows the arm to see and interact with its environment. A tablet-based user interface allows the user to issue commands either through voice input or large touch buttons designed for accessibility. It addresses the following problem statement, from our previous Requirements:

> Identify and implement three unique Activities of Daily Living (ADLs) that can be automated by a Kinova Gen3 robotic arm mounted on a wheelchair, specifically for a user with a physical disability affecting upper-limb mobility. The three tasks are: (1) clear household objects from a table surface, (2) retrieve a fallen water bottle within the arm's range, and (3) support medication administration through identity confirmation and retrieval.

Object detection is performed using AprilTag markers attached to objects, allowing the system to identify and locate objects within the robot's workspace. The system is first implemented and tested in simulation using ROS2 and Gazebo before being deployed on the physical hardware of the robotic arm. We work first to achieve an 80% success rate in simulation before we move to real-world testing.

This specification document describes the modular system architecture, user interface, perception system, motion planning approach, and development plan used to implement the assistive robotic system.

2. Glossary

Action Client: a ROS2 action instance, to send a typed goal message to a server, receive feedback during execution, and retrieve a result after execution. Used in helper_moves to interface with the MoveGroup action server at /move_action.

Activity of Daily Living: ADL; an activity essential to everyday life that are often made more difficult when living with a disability.

AprilTag: a QR code tag that connects to vision package tag36h11, allows for the connection to ROS2 to connect and "see" visible interactable objects. **[3]**

Base_link: the root coordinate frame of the Kinova Gen3 arm, fixed at the center of the arm's mounting position. All object poses, collision objects, and motion goals in the system are expressed in this frame.

Collision Object: A ROS2 message defining a geometric primitive (box, cylinder, sphere) with a pose, in a given coordinate frame. Used to plan obstacles for the arm to avoid during planning.

GetTagPose.srv: A ROS2 service definition in adl_interfaces/srv/. Used to implement a server on a topic name, to track tag poses.

Home Position: a robot configuration in the Kinova Gen3 SRDF file, under the group name "manipulator". Used as a safe resting position before and after tasks, and as recovery in motion failure.

Household Object: a simple rigid object common in daily human life, grabbable by the Kinova Gen3 arm, and with an attached QR code with a unique identification code for each object type. (eg. small box, plastic bottle, cup, medicine bottle). This object should have a bounding box size between about 5 and 15cm and be lighter than 1kg.

Kinova Gen3 Robotic Arm: A lightweight robotic arm designed to safely and precisely pick up and move objects. For the project, the arm is assumed to be attached to one side of a user's wheelchair, for regular use.

Medicine Bottle: a rigid cylindrical pill bottle with a printed label listing a patient's name clearly, and a unique QR code for robot identification.

MoveGroup: the central planning and execution server. Accepts MotionPlanRequest goals, performs collision-aware path planning, and executes approved trajectories on robot controllers.

MotionPlanRequest: A message used to define a specific motion planning problem, when considering known obstacles and constraints.

Physical Disability: An impairment that majorly limits mobility, physical functioning, or dexterity in an individual's daily life.

Planning Scene: the internal MoveIt 2 data structure for all known geometry in the robot's environment. Published at startup by scene_static and the scene vision node, and regularly updated during completion.

Real-Time: updates to object poses should occur at least 5 times per second, less than 100-200 ms.

Table: a flat, horizontal surface, with a height of approximately 45cm, and dimensions of 55cm × 55 cm, existing within the Kinova arm's workspace.

Tool Frame: the end-effector's reference frame, located at the center of the gripper mounting interface.

TF2: the ROS2 transformation library. It maintains a tree of coordinate frame transformations to transform a pose from one frame to another.

Upper-Limb Mobility: Ease of movement in arms, forearms, wrists and hands.

Water Bottle: standard waterbottle (eg. 500-750 mL), with a mass of less than 1kg, lying on its side on the floor.

Wheelchair: a medical device used by a person with a physical disability that affects their mobility, for regular, daily locomotion.

3. Constraints

There are several constraints that influence the design and implementation of the assistive robotic arm system.

1. **Schedule**

- All development and testing must be completed within the course timeline.
- Simulation milestones must precede real hardware testing.
- No ADL task can be tested on the arm before achieving an 80% success rate in simulation.

2. **Hardware**

Parameter	**Constraint**
Robot Arm	Kinova Gen3, 7 DoF, manipulator
Gripper	Robotiq 2F-85, max opening 85 mm (0.085 m)
Gripper Force	5N-220N, from Robotiq 2F-85 datasheet [7]
Gripper Speed	13–101 mm/s, from Robotiq 2F-85 datasheet [7]
Arm Payload	4 kg
Camera	Intel RealSense D415
Camera Resolution	640x480 pixels
Tag Size	5cm x 5cm April Tag
Object Size Range	5-15cm bounding box, <= 1kg

3. **Software**
   - We are working with ROS2 Jazzy on Ubuntu 24.04 LTS.
   - MoveIt2 is necessary for motion planning.
   - The system must be fully operable in simulation before real hardware testing.
   - Simulation uses Gazebo physics engine.
4. **Workspace**

All interactable objects exist within the arm's reachable workspace. The lab space is as follows:

Element	**Dimensions | Position**
Table Surface	0.55 × 0.55 × 0.45 | (0.60, 0.00, 0.225)
Wheelchair Block	0.05 × 0.80 × 0.80 | (-0.30, 0.00, 0.40)
Back Wall	0.05 × 1.00 × 1.20 | (0.95, 0.00, 0.60)
Floor	2.00 × 2.00 × 0.01 | (0.00, 0.00, -0.005)

5. **Perception**
   - AprilTag markers must be on all manipulable objects and destinations.
   - Tags must be visible to the camera from the planned approach direction.
   - Object bounding boxes must be between 5 and 15cm in any dimension.
   - Pose updates occur at 5Hz, with a latency of less than 200 ms (real time, Req 5c)
6. **Safety**
   - All motion plans are checked by MoveIt for collisions against the scene before execution.
   - On task failure, the arm must go to home position, for recovery.
   - The medication hand-off cannot move unless the user's name is confirmed.
   - Real hardware testing can't begin until 80% of simulation is successful.
7. **Accessibility**
   - The UI must require no more than two user actions to initiate any task (Req 1b).
   - The UI must support both voice commands and button presses (Req 1a).
   - Task status must be displayed with test or icons and updated within 2 seconds of state changes (Req 1c).

4. Applicable standards

Standard	Importance to Project
ROS2 REP-103 [6]	Defines standard units of measurement. All numerical values must conform to these units.
ROS2 REP-105 [5]	Defines standard coordinate frames (base_link, map, camera_link, etc.). The base_link is used as the reference for object poses, collision objects, and motion goals.
ISO 10218-1:2011	Defines safety requirements for industrial robots. Informs design requirements for collision avoidance, simulation-first testing strategies, as well as 30% velocity/acceleration limits during ADL tasks.
IEC 62061:2021	Functional safety standard for machinery control ststems. Requires collision detection and safe-stops.
Robotiq 2F-85 Manual [7]	Defines the Robotiq gripper's position, force, and speed ranges.

5. Design

This section describes the system's complete design. It is organized from high-level to low-level.

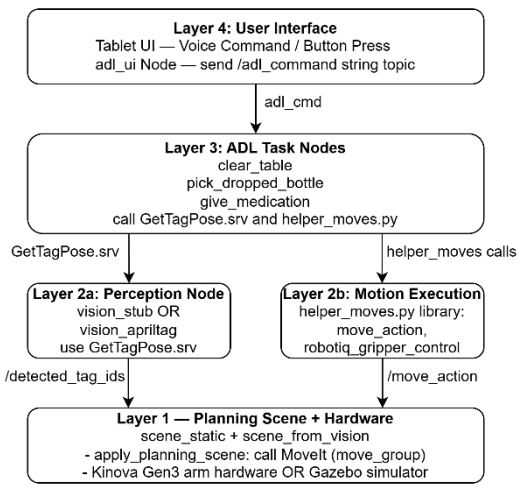


Figure 1: High-Level Project Architecture

5.1. System Overview

The system is structured into four functional layers, where control flows downward from the user to the actual hardware implementation. Information moves upwards from sensors to the task logic. These layers help make sure that each layer can be individually tested and developed. Pictured left, in Figure 1, is a diagram showcasing the project's functional structure in four layers.

5.2. Hardware Configuration

The physical workspace is fixed and known in advance. All collision object dimensions and positions in section 3.3 were measured from the physical lab setup and are hardcoded into scene_static to be avoided. The arm is mounted to the left side of the wheelchair, and the wrist camera is mounted at the end of link 6, with its optical axis aligned with the tool_frame z axis. This workspace is pictured in Figure 2.

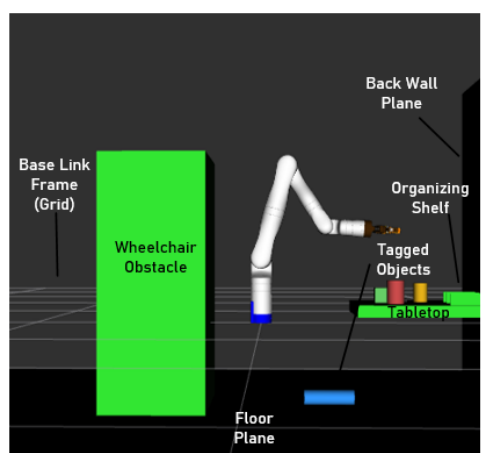


Figure 2: Labeled Robot Workspace

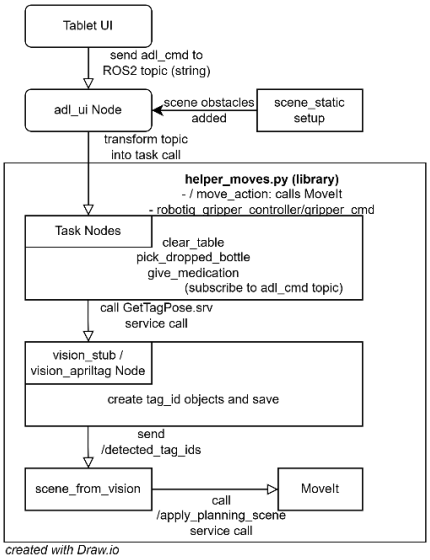


Figure 3: ROS2 Node Flowchart

5.3. Software Architecture

All system components are ROS2 nodes within the adl_tasks and adl_interfaces packages. Helper_moves.py acts as a Python library, rather than a node, to be imported by task nodes when started. Shown in Figure 3.

The two ROS2 packages are structured as follows:

adl_interfaces – custom service definition (GetTagPose.srv). Built as a cmake package, required for service type generation.

adl_tasks – all used Python nodes and libraries

- vision_stub.py, vision_apriltag.py,
- scene_static.py, scene_from_vision.py, helper_moves.py
- clear_table.py, pick_dropped_bottle.py, give_medication.py
- adl_ui.py, apriltag_key.py.

Built as Python packages.

Build and Launch Commands:

colcon build --symlink-install --packages-select adl_interfaces adl_tasks && source install/setup.bash

ros2 launch kinova_gen3_7dof_robotiq_2f_85_moveit_config robot.launch.py robot_ip:=192.168.0.1 use_fake_hardware:=true

 # Then, each in its own terminal, run:

ros2 run adl_tasks scene_static, ros2 run adl_tasks vision_stub, ros2 run adl_tasks clear_table

5.4 Interface and Message Specifications

This section specifies the communication channels within the system as defined in our system: topics, services and actions.

5.4.1 Topics

Topic Name	Message Type	Publisher | Subscriber	Description
/adl_command	std_msgs/String	adl_ui | task nodes	One string defining the specific task to be called. Only triggers execution. Only one task should trigger at once.
/detected_tag_ids	std_msgs/Int32MultiArray	Vision node | task nodes, scene_from_vision	Published at 10Hz. Hata field contains a list of currently visible AprilTag IDs. Stale IDs are to be removed after 5s of no detection.
/wrist_mounted_camera/image	sensor_msgs/Image	Camera driver | vision_apriltag	Raw color image from RealSense D415. Convert for tag scanning with CV bridge. Published at 30Hz.
/joint_states	sensor_msgs/JointState	Ros2_control | all nodes	Current joint positions, velocities, and efforts. Used for MoveIt planning and to verify Home.

/robotiq_gripper_controller/joint_trajectory	trajectory_msgs/JointTrajectory	Helper_moves | gripper control	Used in grab_object to control grasp force and speed.
/picked_ids	std_msgs/Int	Task nodes | scene_from_vision	Published by task nodes after a successful pick. Removes the object as a collision object to prevent phantom collisions.

5.4.2 Services

Action Name	Type	Request / Response
get_tag_pose	Adl_interfaces/GetTagPose	***Request:*** int32 tag_id (currently 0-8) ***Response:*** geometry_msgs/Pose pose, bool success, string message. Server blocks until tag is found or 5 second timeout completes.
/apply_planning_scene	Moveit_msgs/ApplyPlanningScene	***Request:*** moveit_msgs/PlanningScene scene ***Response:*** bool success. Called by scene_static and scene_from_vision.

5.4.3 Actions

Action Name	Type	Goal / Result Used
/move_action	Moveit_msgs/MoveGroup	***Goal:*** MoveGroup.Goal with request and planning options ***Result:*** int error code. Value 1 indicates success.
/robotiq_gripper_controller/gripper_cmd	Control_msgs/GripperCommand	***Goal:*** position, max_effort. ***Result:*** position, effort, stalled, reached_goal. Used by close_gripper and open_gripper in simulation.

5.4.4 Custom Service Definition

The GetTagPose service is defined in adl_interfaces/srv/GetTagPose.srv as:

```
# GetTagPose.srv
# Request
int32 tag_id          # AprilTag ID to look up (0-8)
---
# Response
geometry_msgs/Pose pose    # pose in base_link frame
bool success               # false if tag not found within timeout
string message             # human-readable status or error message
```

5.5 Perception System

The system uses AprilTag (tag36h11) detection to convert raw camera images into 6-DOF object poses in the base_link coordinate frame. The perception system was designed to allow either the stub or the real vision nodes to be tested and to create an identical interface. Task nodes should remain unaware of which is running.

- **INPUT:** /wrist_mounted_camera/image
- **PROCESSING PIPELINE (vision_apriltag):**
    - 1. Convert ROS image message to OpenCV grayscale numpy array.
    - 2. Return list of detection objects, each with a tag_id, pose, center, and corners.
    - 3. For each detection where tag_id is in dictionaries OBJECTS or LOCATIONS:
        - Build the pose, transform to the base_link pose, and cache.
    - 4. Use a timer at 10Hz to purge old tags and publish remaining.

- 5. GetTagPose service handler: validate tag is in the dictionaries and fail else. Return pose if found within 5s, or return failure.

- **OUTPUT:** /detected_tag_ids
- **STUB MODE:** returns hardcoded stub poses as always visible, without camera interface.

<u>Tag Placement Specifications</u>

Object	ID	Tag Placement	Approach Vector	Grasp Offset (m)	Gripper Width (m)	Force (N) | Speed (m/s)
Water Bottle	0	Side (face-up, on floor)	[0, 0, −1] above	[0.05, 0, 0]	0.067 + tol	15 | 0.03
Medication	1	Side (upright, facing robot)	[1, 0, 0] horizontal	[0, 0, 0.04]	0.060 + tol	10 | 0.03
Cup	2	Side (upright, facing robot)	[1, 0, 0] horizontal	[0, 0, 0.04]	0.075 + tol	10 | 0.05
TV Remote	3	Top (face up)	[0, 0, −1] above	[0, 0, 0.02]	0.043 + tol	10 | 0.03
Cube	4	Top (face up)	[0, 0, −1] above	[0, 0, 0.05]	0.065 + tol	10 | 0.05

5.6 Planning Scene Management

The MoveIt planning scene is the main representation of all obstacles, and is initialized with permanent geometry on startup, with dynamically updated objects as detected and picked up. All motion plans are checked against the planning scene before execution, which would reject any plan that would cause a collision.

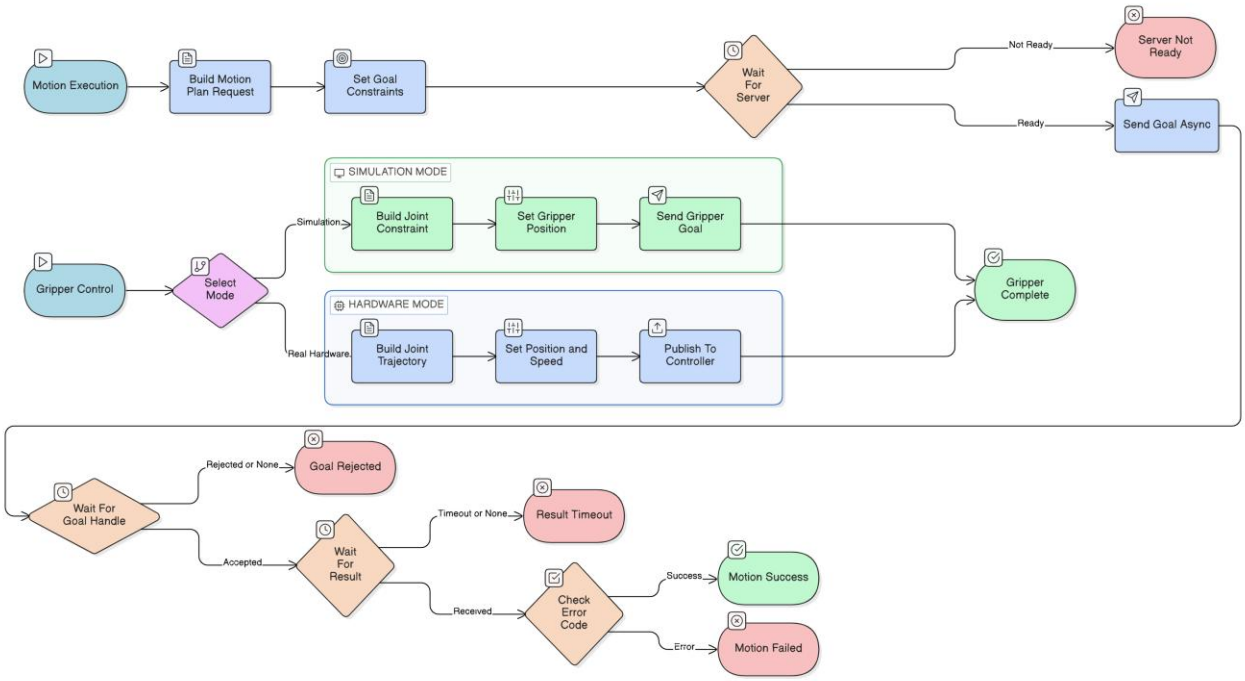
5.7 Motion Planning and Execution

Any arm motion is planned by MoveIt 2, through the OMPL planner suite and the MoveGroup action server. Task nodes user helper_moves.py to call MotionPlanRequests and submit them as MoveGroup action goals. Gripper motion then either uses MoveIt states or real hardware through JointTrajectory publication.

The MoveItHelper class is instantiated once per task node and holds an ActionClient, to apply planning parameters to an arm motion goal:

- Group_name: Planning group for the robotic arm, defined in SRDF
- Num_planning_attempts: number of allowed planning attempts
- Allowed_planning_time: time for planning before move
- Max_velocity_scaling_factor: max speed allowed near a user
- Max_acceleration_scaling_factor: max acceleration allowed. Matched to velocity
- Workspace min: Full reachable workspace
- Workspace max: Upper bound on arm
- Plan_only = false: plan and execute in one call
- Replan: allow automatic replanning if interrupted.
- Pose_link: Target link for cartesian pose.
- Posiiton tolerance: 5mm, precise to grasp, loose for reliable IK.
- Orientation tolerance: First axis to relax if planning fails.

.



5.8 Motion Planning and Execution

Each ADL task is its own independent ROS2 node. All three share the same structure: subscribe to /adl_cmd, wait for matching command string, and execute the task sequence by calling helper_moves.py and GetTagPose, recovering on home after a failure. Each task is defined below:

5.8.1 Clear Table (Requirement 2)

Clear up to three household objects – cup, TV remote, or cube – from the table surface. Each object is grasped and placed at a predefined destination location. The task will continue whether or not an earlier object fails.

- **Inputs:** /adl_command='clear_table', /detected_tag_ids, GetTagPose responses for each ID
- **Outputs:** Arm motions, /picked_ids on each successful pick, log messages for debugging, go_home on completion.
- **Success Criterion:** At least 3 objects cleared in >80% of simulation runs; 70% in real hardware trials.

5.8.2 Pick Up Fallen Water Bottle (Requirement 3)

Detect a water bottle lying on its side on the floor, grasp it at floor level, lift it to the table's height, and place in a retrievable location near the user's edge/

- **Inputs:** /adl_command='pick_dropped_bottle', GetTagPose responses for each ID
- **Outputs:** Arm motions, /picked_ids = [0] on successful pick, log messages for debugging, go_home on completion.
- **Success Criterion:** Detect, grasp, lift, and place in >80% of simulation runs; 70% real hardware.

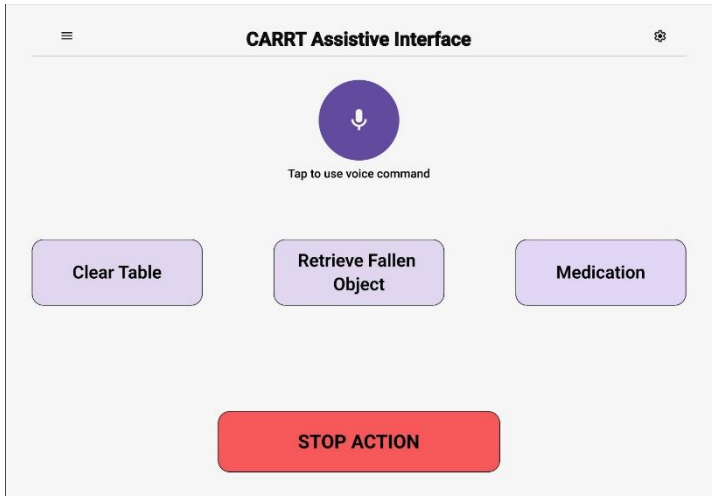
5.8.3 Medication Hand-Off (Requirement 4)

Detects and reads a medication bottle on a table, presents the detected patient's name to the user for confirmation, and hands off the bottle only if the user confirms. If the names do not match, the arm will not move. This is a critical function.

- **Inputs:** /adl_command='give_medication', GetTagPose responses for each ID, user UI response
- **Outputs:** Camera moves and reads label, UI displays detected name and prompts confirmation, arm grasps and delivers only on correct confirmation and UI displays outcome. Arm doesn't move on mismatch.
- **Success Criterion:** Correct match and handoff in 80% of simulation runs. Name mismatch will always abort without motion.

5.9 User Interface

The UI is implemented as a web application (adl_ui/figma_to_ros) served from the ROS2 machine. The user can access it from a tablet browser amd it publishes to /adl_command and displays a live status from the executing task node. In the accompanying figure, the current screen interface is depicted.



6. Risk analysis and mitigation

The development of the assistive robotic system involves several potential risks that may affect system performance, safety, or project completion. These risks are identified early in the design process so that mitigation strategies can be implemented during development and testing.

Risk	Description	Mitigation Strategy
Vision detection failure	AprilTag markers may not be detected due to lighting conditions or camera angle	Use high-contrast tags, maintain stable lighting conditions, and verify detection of reliability in simulation before testing real hardware.
Motion planning failure	MoveIt may fail to generate a valid trajectory due to collisions, unreachable poses, or planning error	Use collision objects in the planning scene and implement recovery behaviors such as returning to the home position.

Simulation vs. real-world differences	Behavior observed in Gazebo simulation may not perfectly match real robot behavior.	Perform extensive simulation testing and gradually transition to real hardware testing after achieving consistent results.
ROS2 communication failure	ROS2 nodes may fail to communicate properly due to incorrect topic names, message types or node crashes	Use modular node testing and ROS2 debugging tools to verify topic communication and service responses.

7. Design trade-off

Each trade-off represents a decision where multiple approaches were seriously considered.

- **Perception Method**
  - ***Considered:*** AprilTag markers, YOLO object detection
  - ***Choice:*** AprilTag markers
  - ***Reasoning:*** AprilTags don't require training while providing the necessary configuration for the project's application, making it the simpler choice.
  - ***Tradeoff:*** less flexibility, need for tags on any interactable object
- **MoveIt Python**
  - ***Considered:*** moveit_py, directly use moveit_msgs
  - ***Choice:*** use direct action client
  - ***Reasoning:*** moveit_py is not used in the Jazzy binary install and could not be built from source.
  - ***Tradeoff:*** harder to code than moveit_py
- **Motion Planning**
  - ***Considered:*** free-space only planning, hybrid (free space / cartesian)
  - ***Choice:*** Hybrid
  - ***Reasoning:*** Hybrid allows for planning less trajectories that may cause collisions through strange contortions. Cartesian used specifically to descend on objects.
  - ***Tradeoff:*** not the most predictable or the most robust
- **Scene Representation**
  - ***Considered:*** static scene / real-time vision / both
  - ***Choice:*** both combined
  - ***Reasoning:*** hardcode static/known obstacles while reading QR tags from vision nodes.
  - ***Tradeoff:*** static object must be updated when static scene changes.
- **Simulation Environment**
  - ***Considered:*** Gazebo / Rviz only / Real hardware
  - ***Choice:*** Full Gazebo simulation
  - ***Reasoning:*** Full Gazebo allows more robust training, though Rviz is more stable. Both will be used, with Rviz in development stages and Gazebo in greater Demos.
  - ***Tradeoff:*** Different coding paths are needed for different sims and hardware types.

8. Project plan

- Week 1 – 1/12 – Course Begins
- Week 2 – 1/19 – Establish Company Contact and Requirements/Expectations
    - Lock down the three ADL tasks: (currently)
        - Pick something up from the floor
        - Clear/organize the table
        - Flip a light switch on/off
    - User Stories:
        - "As a daily user, I want a robotic arm that can grab my water bottle from the floor, so that its easily retrievable if knocked over."
        - "As a daily user, I want a robotic arm that can clear an area of a table, so that it is then available for another use."
        - "As a daily user, I want a robotic arm that can turn on or off a light, so that I don't need to ask for help."
    - Determined QR Recognition for Perception
    - Weekly Status Reports to CARRT
- Development Begins
- Week 3 – 1/26 – Solidify Requirements, Begin UI Planning / QR Code Recognition
    - Determine steps to implement each ADL task, and success criteria.
    - Draft requirements document
    - Begin UI implementation planning
    - Prepare base QR recognition for demo
    - **Status Report / Demo Due 1/30**
- Week 4 – 2/2 – Continue Week 3, Solidify UI and QR Code Architecture
    - Finalize UI Layout
    - Finalize QR code rules and camera integration
    - Start QR detection prototype
    - **Submit Refined Requirements Document 2/6**
- Week 5 – 2/9 – Build System Skeleton
    - Begin integrating QR object detection and pose estimation for tasks.
    - **Status Report / Demo Due 2/13**
- Week 6 – 2/16 – Add Task-Specific Behavior
    - Create support for the 3 ADL tasks in the code.
    - Create light switch detection/interaction logic.
    - Improve QR detection as needed.
    - **Prepare: Design Peer Review Presentation 2/20**
- Week 7 – 2/23 – Review and Refinement
    - Use Peer/Professor feedback to improve design
        - ADLs, implementation, etc.
- ➢ *Week 8 – 3/2 – Finalize Specification*
    - ✓ *Finalize Design / Logic / UI*
    - ✓ *Document ADLs and component usage.*
    - ✓ ***Status Report / Demo Due 3/6***
    - ✓ ***Submit Specifications Document 3/6***
- Week 9 – 3/9 – Testing and Evaluation Planning

    - Design/Complete Test Plan
    - Determine Metrics for Refinement
    - **Submit Test Plan Document 3/13**
- *Week 10 – 3/16 – Spring Break*
    - *Spring Break*
- Week 11 – 3/23 – Ensure Demo Viability
    - Ensure each task functions properly.
    - Demonstrate each ADL task.
    - Begin optimization.
    - **Demo Due 3/27, Schedule with Professor**
- Week 12 – 3/30 – Optimization Phase Begins
    - Improve design and functionality (consistency)
    - Reduce failure cases.
    - Begin the final data collection.
    - Start drafting the final report.
    - Begin drafting for publication, with collected data.
- Week 13 – 4/6 – Performance Check / Data Analysis
    - Optimize accuracy and reliability.
    - Analyze data for distinguishable patterns.
    - Finish and begin editing of the paper for publication.
    - **Status Report / Demo Due 4/10**
- Week 14 – 4/13 – Final Report Drafting / Figure Creation
    - Finalize report elements
    - Finalize paper for publication.
    - Begin design of other final submission elements.
- Week 15 – 4/20 – Final Draft
    - **Finish Paper for Publishing**
    - **Status Report / Demo Due 4/24**
- Week 16 – 4/27 – Final Submission Due
    - Final Demonstration and Presentation
    - Check over final deliverables
    - Submit complete project
    - **Final Presentation Due 5/1**

9. Traceability to Requirements

Requirement	5.2 HW	5.4 I/F	5.5 Percep.	5.6 Scene	5.7 Motion	5.8 Tasks	5.9 UI	6 Risk
Req. 1 — Accessible UI							✓	
Req. 1a — voice+button							✓	
Req. 1b — ≤2 actions							✓	

Req. 1c — 2s status update							✓	
Req. 2 — Clear Table	✓	✓	✓	✓	✓	✓		✓
Req. 2a — identify ≥3 items			✓	✓		✓		
Req. 2b.i — grasp item					✓	✓		
Req. 2b.ii — move to drop				✓	✓	✓		
Req. 2c — 80% sim success						✓		✓
Req. 2d — 70% real success						✓		
Req. 3 — Pick Bottle	✓	✓	✓	✓	✓	✓		✓
Req. 3a — detect floor pose			✓			✓		
Req. 3b — grasp + lift					✓	✓		
Req. 3c — place on table					✓	✓		
Req. 3d — 80% sim success						✓		✓
Req. 3e — 70% real success						✓		
Req. 4 — Medication	✓	✓	✓		✓	✓	✓	✓
Req. 4a — detect + read label			✓			✓		
Req. 4b — UI confirm name						✓	✓	
Req. 4c.i — grasp bottle					✓	✓		
Req. 4c.ii — move to handover					✓	✓		
Req. 4d — abort on mismatch						✓	✓	✓
Req. 4e — 80% sim success						✓		

Req. 5 — QR perception			✓					✓
Req. 5a — visible at range	✓		✓					
Req. 5b — 80% ID accuracy			✓					✓
Req. 5c — real-time ≥5 Hz		✓	✓					
Req. 6 — Sim first				✓				✓
Req. 6a — ROS2 + Gazebo + env	✓	✓		✓				
Req. 6b — all ADLs in sim						✓		
Req. 6c — env objects in sim	✓			✓				

10. Traceability to needs and factors

Requirements	Applicable	Not Applicable	Description of Application
Public Health Needs	✓		Public health needs are addressed by reducing the need for disabled users to over-exert themselves or ask for outside assistance which the robotic arm provides. **Relevant Design Sections: 5.8.1, 5.8.2, 5.8.3**
Public Safety Needs	✓		Public safety is addressed by ensuring that the robot achieves at least an 80% success rate during simulation before being used with the real equipment. **Relevant Design Sections: 5.6, 5.7, 5.8.3, 6**
Public Welfare Needs	✓		Through assisting with Activities of Daily Living (ADLs) as outlined in requirements (2), (3), and (4), the robotic arm aims to enhance the quality of life for individuals with disabilities. **Relevant Design Sections: 5.8, 5.9**

Global Factors		✓	Global factors are not addressed by the project, as the goal is more localized. The system doesn't involve global effects, since it is only applicable as an assistive robot for those with disabilities. **Relevant Design Sections: N/A**
Cultural Factors		✓	Cultural factors are not addressed by this project. The robotic arm works to provide basic assistance in daily tasks present over the world and is typically unaffected by a user's culture. **Relevant Design Sections: N/A**
Social Factors	✓		Social factors are addressed through the focus on the goal of independence and improved accessibility. **Relevant Design Sections: 5.8, 5.9**
Environmental Factors		✓	Environmental factors are not addressed as the robotic arm does not generate significant waste. **Relevant Design Sections: N/A**
Economic Factors	✓		Economic factors are addressed by using open-source software that significantly reduces the cost of development. **Relevant Design Sections: 7 (simulation trade-offs), 8**

References

[1] Kinovarobotics, "ros2_kortex: ROS 2 driver for the Gen3 Kinova robot arm," GitHub Repository. [Online]. Available: https://github.com/Kinovarobotics/ros2_kortex (Accessed: Mar. 2026).

[2] PickNik Robotics, "MoveIt 2 Documentation," 2024. [Online]. Available: https://moveit.picknik.ai/main/index.html (Accessed: Mar. 2026).

[3] E. Olson, "AprilTag: A robust and flexible visual fiducial system," in Proc. IEEE Int. Conf. Robotics and Automation (ICRA), Shanghai, China, 2011, pp. 3400–3407. tag36h11 image set: https://github.com/AprilRobotics/apriltag-imgs

[4] Open Robotics, "ROS 2 Jazzy Jalisco Documentation," 2024. [Online]. Available: https://docs.ros.org/en/jazzy/ (Accessed: Mar. 2026).

[5] W. Meeussen, "REP-105: Coordinate Frames for Mobile Platforms," ROS Enhancement Proposal 105, 2010. [Online]. Available: https://www.ros.org/reps/rep-0105.html

[6] W. Garage, "REP-103: Standard Units of Measure and Coordinate Conventions," ROS Enhancement Proposal 103, 2010. [Online]. Available: https://www.ros.org/reps/rep-0103.html

[7] Robotiq Inc., "2F-85 and 2F-140 Instruction Manual," Rev. L, 2021. [Online]. Available: https://robotiq.com/support/2f-85-adaptive-robot-gripper

[8] Intel Corporation, "Intel RealSense D415 Datasheet," 2019. [Online]. Available: https://www.intelrealsense.com/depth-camera-d415/

[9] International Organization for Standardization, "ISO 10218-1:2011 — Robots and Robotic Devices — Safety Requirements for Industrial Robots — Part 1: Robots," 2011.

[10] International Electrotechnical Commission, "IEC 62061:2021 — Safety of Machinery — Functional Safety of Safety-Related Control Systems," 2021.